SQL Performance Challenge - Retail Bank

Accounts, payments and loans of a retail bank: write correct SQL on a large dataset, then make it fast

5 analytical questions | dataset scale **large** (~46 M transactions)

Generated 2026-10-06

Name _____

Date _____

Total time spent _____

Instructions

You get the core-banking database of an Indian retail bank (transactions from January 2024 to December 2025, loan repayment history since 2019) with tens of millions of rows. For each of the 5 questions you must:

1. Write a query that gives the **correct** result.
2. Make that analysis run in **as little time as possible**, with exactly the same result.

Dataset size (scale `large`): about 46,000,000 transactions on 950,000 accounts and 5,000,000 loan instalments.

Files in your folder

File	Purpose
`bank.db`	SQLite database with all the data
`SQL_Challenge_Questions.pdf`	This question paper
`run_query.py`	Runs a `.sql` file, times it and saves the result
`answers/`	Your SQL files: `q1_v1.sql` (first version), `q1_v2.sql` (next version), ...
`results/`	Query results (CSV) and `timing_log.csv`, written by `run_query.py`

Running your SQL

- `python run_query.py answers/q1_v1.sql` - run, time and save the result to `results/q1_v1.csv`
- `python run_query.py answers/q1_v1.sql --plan` - show the execution plan without running the query
- `python run_query.py answers/q1_v1.sql --timeout 300` - stop after 300 seconds (default 900)
- `python run_query.py -e "SELECT COUNT(*) FROM transactions"` - run one statement typed on the command line
- `python run_query.py --list-objects` - list the database objects you have created
- `python run_query.py --reset` - remove everything you created (back to the original database)
- You may also use DB Browser for SQLite, DBeaver or the `sqlite3` command line.
- Prefer another database (PostgreSQL, MySQL, ...)? Run the generator again with `--csv` to get every table as a CSV file.

A `.sql` file may contain several statements separated by `;`. Each statement is timed separately.

Rules and what to submit

- Do not change or delete the original data.
- Anything else you create in the database is allowed, but the time to create it counts: report it as **setup time**, separately from the **query time**.
- All versions of a question must return identical results. A fast wrong answer scores zero.

- Before re-timing an earlier version, run `python run_query.py --reset` so that objects created later do not change its time.
- For every question submit: all versions (`qN_v1.sql`, `qN_v2.sql`, ...) with their times (or "stopped after N s"), the result CSV, and a short note: why was the first version slow and what made the later version faster?
- Fill in the "Your timings" table under each question.

The Questions

Common definitions (used in all questions)

- Only transactions with `status = 'success'` change money. `failed` and `reversed` transactions are ignored everywhere.
- `amount` is always positive; `direction` says whether money came in (`credit`) or went out (`debit`).
- **Customer-initiated** transaction = channel is `upi`, `card_pos`, `atm`, `imps`, `neft` or `branch` (not `system` and not `auto_debit`).
- Times are text `YYYY-MM-DD HH:MM:SS`; dates are `YYYY-MM-DD`. **Days between** two values = `julianday(later) - julianday(earlier)`.
- Round money and percentages to **2 decimals** in the final output only.

What you must do for every question

- **Part A - Correct result.** Write a query that returns exactly the requested columns, in the requested order. Your row count must match the expected rows.
- **Part B - Fast result.** Run your query with `run_query.py` and write down the time. Then make the same analysis take as little time as you can. The result must stay exactly the same. Save every version (`q1_v1.sql`, `q1_v2.sql`, ...) and record its time in the table under the question.

Q1. Month-end balances

Business question: at the end of every month of 2025, how much money sat in each type of account, and how many accounts were overdrawn?

- An account's **balance** at a moment = `opening_balance` + all successful credits - all successful debits of that account with `txn_time` up to that moment.
- **Month end** = the last second of the month (`YYYY-MM-DD 23:59:59` of the last day).
- An account counts for a month end if its `open_date` is on or before that month end - **also if it had no transaction in that month** (its balance simply carries over).
- For each month of 2025 and each `account_type`: `accounts`, `total_balance`, `avg_balance`, `overdrawn_accounts` (balance below 0) and `overdrawn_amount` (sum of the negative balances, shown as a positive number).

Example: opening balance 10000 (opened 2023), credit 5000 on 2025-01-10, debit 20000 on 2025-03-02. Month-end balances: Jan 15000, Feb 15000, Mar -5000 (overdrawn by 5000), Apr -5000, ...

Output columns: `month, account_type, accounts, total_balance, avg_balance, overdrawn_accounts, overdrawn_amount`

Order by: `month, account_type`

Expected rows: 36 (12 months x 3 account types)

Your timings

Ver.	File	Setup time (s)	Query time (s)	What changed / why faster
v1				
v2				
v3				

Q2. Rapid-fire debits

Business question: which accounts show card/UPI spending bursts typical of a stolen card or phone?

- Use successful debits with channel `upi` or `card_pos`.
- For every such debit d, its **window** is d itself plus all such debits of the same account with `txn_time` from d.txn_time up to **600 seconds later** (inclusive).
- `debits_in_window` = number of debits in the window; `amount_in_window` = their total amount.
- For each account keep its best window: highest debits_in_window, then highest amount_in_window, then earliest window_start (= d.txn_time).
- Return the 30 accounts with the best windows: order by debits_in_window DESC, amount_in_window DESC, account_id.

Example: debits at 10:00:00, 10:04:00, 10:09:59 and 10:10:30. The window of the first debit contains 3 debits (10:10:30 is more than 600 seconds later); the window of the second contains 3.

Output columns: `account_id, customer_id, account_type, window_start, debits_in_window, amount_in_window`

Order by: `debits_in_window DESC, amount_in_window DESC, account_id`

Expected rows: 30

Your timings

Ver.	File	Setup time (s)	Query time (s)	What changed / why faster
v1				
v2				
v3				

Q3. Loan delinquency buckets

Business question: at every month end of 2025, how many loans of each product were current or 1-30, 31-60, 61-90, 90+ days past due, and how much money was overdue?

- Month end = last day of the month (a date, e.g. `2025-02-28`).
- A loan is **active** at a month end if its `sanction_date` is on or before the month end **and** it is not finished: fewer than `tenure_months` of its instalments were due and paid by that month end (due_date and paid_date both on or before the month end).
- An instalment is **unpaid at month end** if its `due_date` is on or before the month end and it was not paid by then (`paid_date` is NULL or later than the month end).
- **DPD** (days past due) of a loan = days between the month end and the `due_date` of its oldest instalment that is unpaid at that month end. No such instalment, or DPD = 0 -> the loan is **current**.
- `overdue_amount` = sum of `amount_due` of all instalments unpaid at that month end.
- Count every active loan in exactly one of: `current`, `dpd_1_30`, `dpd_31_60`, `dpd_61_90`, `dpd_90_plus`.

Example: month end 2025-03-31; instalments due 2025-01-15 (paid 2025-04-10), 2025-02-15 (unpaid), 2025-03-15 (paid 2025-03-14). Unpaid at month end: the January and February instalments -> DPD = 75 (counted as 61-90), overdue_amount = two instalments.

Output columns: `month_end, product, active_loans, current, dpd_1_30, dpd_31_60, dpd_61_90, dpd_90_plus, overdue_amount`

Order by: `month_end, product`

Expected rows: 48 (12 month ends x 4 products)

Your timings

Ver.	File	Setup time (s)	Query time (s)	What changed / why faster
v1				
v2				
v3				

Q4. Round-trip transfers

Business question: where did money go from one account to another and come back almost unchanged within two days (a common money-laundering pattern)?

- An **internal transfer out** is a successful debit with a `counterparty_account_id`.
- A transfer out from account A to account B of amount X at time t1 is **returned** by a successful debit from B with counterparty A at time t2 when: t1 < t2 <= t1 + 48 hours, the returned amount is between 98 % and 102 % of X, and X >= 50000.
- If several debits return the same transfer, use the earliest one.
- `hours_between` = (t2 - t1) in hours.
- Return the 40 round trips with the largest original amount.

Example: A -> B 200000 on 2025-03-01 10:00, B -> A 201500 on 2025-03-02 18:00 (100.75 %, 32 hours) is a round trip. B -> A 150000 would not be (75 %).

Output columns: `from_account, to_account, sent_at, returned_at, sent_amount, returned_amount, hours_between`

Order by: `sent_amount DESC, from_account, sent_at`

Expected rows: 40

Your timings

Ver.	File	Setup time (s)	Query time (s)	What changed / why faster
v1				
v2				
v3				

Q5. Dormant accounts that suddenly move money

Business question: which savings accounts were silent for at least 180 days and then moved a lot of money out within a week of waking up?

- Use **savings** accounts and only their successful **customer-initiated** transactions (credits and debits).
- Sort each account's customer-initiated transactions by time. When two consecutive ones are at least 180 days apart, the later one is a **reactivation**; `dormant_days` = days between them (2 decimals).
- `debits_7d` = total amount of successful customer-initiated debits from the reactivation time up to 7 days later (inclusive); `debit_count_7d` = how many.
- Report reactivations with `debits_7d >= 100000`, the 40 with the highest `debits_7d`.

Example: last activity 2024-02-10, next activity 2024-09-01 (204 days later) = reactivation; debits of 60000 on 2024-09-02 and 50000 on 2024-09-05 -> debits_7d = 110000 -> reported.

Output columns: `account_id, customer_id, last_activity_before, reactivated_at, dormant_days, debit_count_7d, debits_7d`

Order by: `debits_7d DESC, account_id`

Expected rows: 40

Your timings

Ver.	File	Setup time (s)	Query time (s)	What changed / why faster
v1				
v2				
v3				

Bonus question (optional)

Pick one question. Explain, using the output of `run_query.py --plan` for your first and your fastest version, why the fastest version needs less time. Then describe what you would change in the design of this database if these 5 reports had to be produced every morning.

Schema reference

Times are TEXT `YYYY-MM-DD HH:MM:SS`, dates `YYYY-MM-DD`. Money is in INR. Arrows (->) show which table a column refers to.

branches

column	type	notes
branch_id	INTEGER PK	
branch_name, city, state	TEXT	
opened_date	TEXT	

customers

column	type	notes
customer_id	INTEGER PK	
full_name	TEXT	
birth_year	INTEGER	
city, state	TEXT	
segment	TEXT	retail / priority / nri / business
kyc_level	TEXT	full / minimum / video
join_date	TEXT	

accounts

column	type	notes
account_id	INTEGER PK	
customer_id	INTEGER	-> customers (a customer can have several accounts)
branch_id	INTEGER	-> branches
account_type	TEXT	savings / salary / current
open_date	TEXT	
opening_balance	REAL	balance on the open date, before any transaction in this table
overdraft_limit	REAL	how far a current account may go below zero

merchants

column	type	notes
merchant_id	INTEGER PK	
merchant_name, category, city	TEXT	

transactions (January 2024 - December 2025)

column	type	notes
txn_id	INTEGER PK	not in time order
account_id	INTEGER	-> accounts
txn_time	TEXT	
direction	TEXT	credit / debit
amount	REAL	always positive
channel	TEXT	upi / card_pos / atm / imps / neft / branch / auto_debit / system
merchant_id	INTEGER	-> merchants, for some card/UPI debits
counterparty_account_id	INTEGER	-> accounts, for transfers between two accounts of this bank (both sides are recorded)
status	TEXT	success / failed / reversed

loans

column	type	notes
loan_id	INTEGER PK	
customer_id	INTEGER	-> customers
product	TEXT	home / personal / auto / gold
sanction_date	TEXT	
principal, annual_rate	REAL	
tenure_months	INTEGER	
emi_amount	REAL	monthly instalment

loan_installments (all instalments due up to 2025-12-31)

column	type	notes
installment_id	INTEGER PK	
loan_id	INTEGER	-> loans
installment_no	INTEGER	1, 2, 3, ...
due_date	TEXT	
amount_due	REAL	
paid_date	TEXT	NULL = not paid (as of 2025-12-31)
amount_paid	REAL	NULL when not paid